ThingIndex: Plataforma Open Source para Inteligência Documental com Baseline Relacional

Charles Eduardo Mello Guimarães[1]

[1] Instituto Federal do Paraná (IFPR) – Campus Palmas
Curso de Sistemas de Informação
Palmas – PR – Brasil

charles.eduardo.mello.guimaraes@hotmail.com

Resumo. *Este artigo apresenta o ThingIndex, uma plataforma open source voltada a inteligência documental, organização de coleções e recuperação de contexto com base em modelagem relacional. O trabalho parte do problema de que fluxos documentais frequentemente se apoiam em ferramentas fragmentadas, com baixa auditabilidade e pouca rastreabilidade do contexto reconstruído para busca, indexação e interação assistida por IA. O objetivo geral é propor e especificar o ThingIndex como uma plataforma open source de inteligência documental baseada em modelagem relacional, capaz de organizar, processar, indexar e recuperar contexto documental de forma auditável. Dessa forma, o artigo descreve as motivações, a relevância, os objetivos e o planejamento arquitetural do sistema, estabelecendo a base conceitual e metodológica para o desenvolvimento e a avaliação futura do projeto.*

Palavras-chave: inteligência documental, recuperação de informação, RAG relacional, baseline relacional, open source.

Abstract. *This paper presents ThingIndex, an open-source platform focused on document intelligence, collection organization, and context retrieval based on relational modeling. The work starts from the problem that document workflows often rely on fragmented tools, with low auditability and limited traceability of the reconstructed context for search, indexing, and AI-assisted interaction. The general objective is to propose and specify ThingIndex as an open-source document intelligence platform based on relational modeling, capable of organizing, processing, indexing, and retrieving document context in an auditable manner. Thus, the paper describes the motivations, relevance, objectives, and architectural planning of the system, establishing the conceptual and methodological basis for the future development and evaluation of the project.*

Keywords: document intelligence, information retrieval, relational RAG, relational baseline, open source.

1. Introdução

Em ambientes organizacionais, a produção de documentos, como relatórios técnicos, atas de reunião, contratos e manuais de processos, é contínua. Esses documentos são frequentemente armazenados em sistemas de arquivos, diretórios compartilhados ou plataformas

de gestão eletrônica de documentos sem qualquer forma de estruturação semântica de seu conteúdo. Quando um colaborador precisa localizar uma informação específica, como uma cláusula contratual, um parecer técnico ou uma decisão registrada em ata, a busca geralmente se limita a palavras-chave associadas ao nome do arquivo ou, em alguns casos, a metadados cadastrais. Estudos sobre recuperação de informação em ambientes corporativos indicam que a dependência de mecanismos baseados em palavras-chave e metadados pode dificultar a localização eficiente de documentos relevantes e restringir o acesso ao contexto associado às informações recuperadas [Schymik et al. 2015]. Como consequência, a rastreabilidade da informação recuperada, sua origem documental e a preservação de seu contexto nem sempre são explicitamente suportadas pelos mecanismos tradicionais de busca.

Esse cenário compromete a auditabilidade dos processos, reduz a confiabilidade das informações recuperadas e limita a reutilização do conhecimento organizacional. Em setores como jurídico, compliance e gestão de contratos, nos quais a rastreabilidade constitui um requisito essencial, a ausência de mecanismos capazes de preservar a estrutura documental e a proveniência das informações pode representar riscos operacionais significativos.

Diante desse cenário, define-se a seguinte pergunta de pesquisa: é possível sustentar um sistema de inteligência documental com auditabilidade utilizando exclusivamente modelagem relacional, sem depender de bancos vetoriais ou grafos como camada primária de recuperação?

Para investigar essa questão, propõe-se o ThingIndex, um sistema de inteligência documental fundamentado em uma arquitetura relacional. O projeto organiza coleções, documentos, páginas, parágrafos, tabelas, metadados e artefatos derivados em uma estrutura persistente e rastreável, apoiada principalmente em PostgreSQL. Dessa forma, a relevância da proposta está associada à crescente demanda por soluções capazes de tratar documentos de forma estruturada, segura e auditável.

Além de sua relevância prática, o projeto apresenta interesse acadêmico ao integrar conceitos de engenharia de software, arquitetura de sistemas, modelagem de dados, processamento assíncrono e recuperação de informação, passível de implementação, avaliação e evolução.

Portanto, este documento tem como propósito a qualificação da proposta de pesquisa. São apresentados nas próximas seções os objetivos, a fundamentação teórica, os trabalhos relacionados, a arquitetura proposta, a metodologia de avaliação e o cronograma de desenvolvimento. A implementação do sistema será objeto do TCC II.

2. Objetivos

O presente estudo tem como objetivo geral propor e especificar o ThingIndex como uma plataforma open source de inteligência documental baseada em modelagem relacional, capaz de organizar, processar, indexar e recuperar contexto documental de forma auditável.

2.1. Objetivos específicos

- identificar o problema de fragmentação e baixa rastreabilidade em fluxos de processamento documental;

- definir uma estrutura relacional para representar coleções, documentos, páginas, parágrafos, tabelas, metadados e artefatos derivados;
- projetar uma arquitetura full-stack;
- definir critérios objetivos de avaliação que permitam verificar auditabilidade, precisão na recuperação e custo operacional;
- estabelecer uma *baseline* funcional e avaliável que sustente comparações futuras com abordagens vetoriais ou orientadas a grafo.

3. Referencial Teórico

Esta seção apresenta os fundamentos teóricos que sustentam as áreas de conhecimento envolvidas no projeto do ThingIndex.

3.1. Inteligência Documental e Gestão do Conhecimento

A gestão do conhecimento estuda como organizações criam, retêm e compartilham conhecimento. Nonaka e Takeuchi [Nonaka e Takeuchi 1995] classificam o conhecimento em duas categorias: tácito, que reside na mente das pessoas e é difícil de articular; e explícito, registrado em documentos, manuais e bases de dados. A transição entre eles ocorre por meio da espiral do conhecimento, composta por quatro etapas: socialização, externalização, combinação e internalização. A externalização, conversão do conhecimento tácito em explícito, é a etapa mais relevante para sistemas documentais, pois é o momento em que a experiência individual se registra em um documento acessível a outros.

Davenport e Prusak [Davenport e Prusak 1998] argumentam que o valor do conhecimento está em sua capacidade de ser localizado e reutilizado. Sem mecanismos de organização e recuperação, o conhecimento explícito acumulado se perde. A inteligência documental emerge como a capacidade de um sistema de extrair, organizar e recuperar conhecimento a partir de documentos, preservando sua estrutura semântica, como parágrafos, tabelas, metadados e hierarquias, permitindo que o conhecimento seja localizado por conteúdo, não apenas por nome de arquivo.

Os estudos de Nonaka e Takeuchi [Nonaka e Takeuchi 1995] e de Davenport e Prusak [Davenport e Prusak 1998] fundamentaram a decisão central do ThingIndex de tratar documentos não como arquivos monolíticos, mas como conjuntos de unidades de conhecimento extraíveis. Em vez de armazenar PDFs inteiros e buscar por nome de arquivo, o que ignoraria a espiral do conhecimento ao manter o conhecimento em estado bruto e não combinável, o sistema fragmenta cada documento em parágrafos, tabelas e metadados, armazenados em tabelas relacionais tipadas. Essa abordagem organiza o conhecimento explícito para que ele possa ser localizado, combinado e reutilizado, materializando na prática a etapa de combinação descrita por Nonaka e Takeuchi.

3.2. Recuperação de Informação e Indexação Semântica

Recuperação de Informação (RI) é a área que estuda como encontrar documentos relevantes a partir de uma necessidade de informação [Manning et al. 2008]. Diferentemente de uma consulta a um banco de dados tradicional, a RI lida com texto não estruturado e relevância aproximada.

A área de RI pode ser dividida em duas grandes abordagens: os métodos clássicos, baseados em estatística textual, e os métodos neurais, baseados em vetores densos e representações contextuais.

O modelo clássico mais influente é o TF-IDF (Term Frequency: Inverse Document Frequency), proposto por Salton [Salton 1989]. O TF-IDF calcula um peso para cada termo em cada documento: a frequência do termo no documento (TF) multiplicada pelo inverso da frequência do termo no corpus (IDF). Termos frequentes em um documento, mas raros no corpus geral, recebem peso alto. Robertson e Zaragoza [Robertson e Zaragoza 2009] aperfeiçoaram essa abordagem com o BM25, um modelo probabilístico que incorpora saturação de frequência de termo e normalização por tamanho do documento.

A limitação central dos métodos clássicos é o matching léxico: eles só encontram correspondência exata entre termos da consulta e termos do documento. Uma busca por "automóvel" não encontra documentos que mencionam "carro". Para superar essa limitação, surgiram os métodos neurais, que representam palavras como vetores densos em um espaço contínuo. Com o Word2Vec [Mikolov et al. 2013], palavras com significado semelhante, como "carro" e "automóvel", passaram a ocupar posições próximas nesse espaço vetorial. O BERT [Devlin et al. 2019] avançou ao gerar embeddings contextuais: a palavra "banco" tem vetores diferentes em "banco de dados" e "banco do parque". A similaridade de cosseno entre esses vetores permite buscar por sentido, não por palavra exata, formando a base de sistemas modernos de busca e de arquiteturas como o Retrieval-Augmented Generation (RAG), apresentada a seguir.

3.3. Retrieval-Augmented Generation

Modelos de linguagem de grande escala (Large Language Models - LLMs) como GPT, Llama e Gemini têm conhecimento limitado aos dados com os quais foram treinados. Quando confrontados com perguntas sobre documentos específicos, tendem a alucinar, isto é, gerar respostas confiantes porém incorretas [Ji et al. 2023].

O Retrieval-Augmented Generation (RAG) é uma arquitetura proposta por Lewis et al. [Lewis et al. 2020], pesquisadores do Facebook AI Research (FAIR), no artigo "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". A ideia central é combinar recuperação de informação com geração de texto: um retriever busca trechos relevantes em uma base de conhecimento externa, e um generator (LLM) recebe esses trechos como contexto adicional junto com a pergunta do usuário para gerar uma resposta fundamentada.

O pipeline típico do RAG funciona em cinco etapas:

1. Ingestão: o documento original é recebido pelo sistema.
2. Chunking: o documento é dividido em pedaços (chunks), geralmente de 256 a 1024 tokens (unidades mínimas de processamento de texto utilizadas pelos modelos de linguagem), com ou sem sobreposição.
3. Indexação vetorial: cada chunk é convertido em vetor denso (embedding) e armazenado em um banco de dados vetorial (vector DB), como Pinecone, Weaviate ou pgvector.
4. Consulta: a pergunta do usuário é convertida no mesmo espaço vetorial; o vector DB retorna os K chunks mais similares por similaridade de cosseno.
5. Geração: os chunks recuperados são concatenados com a pergunta e enviados ao LLM, que gera uma resposta com base no contexto fornecido.

A principal vantagem do RAG é que o conhecimento pode ser atualizado sem retreinar o modelo. No entanto, o RAG convencional apresenta limitações significativas: o chunking é geralmente cego (baseado em contagem de tokens, sem respeitar a estrutura semântica do documento); a recuperação por similaridade vetorial é uma caixa-preta, sem explicação do critério de matching; e a auditabilidade é limitada: rastrear de que documento, página e parágrafo veio cada informação exige mecanismos adicionais que a maioria das implementações não oferece.

O estudo do RAG, especialmente o artigo seminal de Lewis et al. [Lewis et al. 2020] e das limitações documentadas por Ji et al. [Ji et al. 2023] sobre alucinação, foi o principal motivador para a proposta do ThingIndex. Identificou-se que a arquitetura RAG resolve o problema de contexto, mas introduz um novo problema de opacidade. Dessa forma, a proposta é substituir o vector DB por uma baseline relacional auditável, isto é, em vez de fragmentar documentos em chunks genéricos e indexá-los por similaridade vetorial, o sistema preserva a estrutura semântica original em tabelas relacionais tipadas (detalhada na Seção 6.7).

3.4. Modelagem Relacional Aplicada a Sistemas Documentais

O modelo relacional foi proposto por Codd [Codd 1970] no artigo "A Relational Model of Data for Large Shared Data Banks". Nele, os dados são organizados em relações (tabelas). Cada linha é uma tupla e cada coluna é um atributo. A consistência dos dados é garantida por três mecanismos:

- Chaves e integridade referencial: cada tupla é identificada por uma chave primária. Referências entre tabelas são feitas por chaves estrangeiras, que apontam para chaves primárias de outras tabelas, garantindo que não existam referências órfãs.
- Normalização: para evitar redundância, os dados são organizados em formas normais. A primeira forma normal (1FN) exige colunas atômicas; a segunda (2FN) exige dependência total da chave; a terceira (3FN) elimina dependências transitivas entre colunas não-chave [Elmasri e Navathe 2016].
- Propriedades ACID: transações em bancos relacionais garantem atomicidade (tudo ou nada), consistência (estado válido), isolamento (transações simultâneas não interferem) e durabilidade (dados persistidos mesmo em falha) [Date 2004].

Buneman et al. [Buneman et al. 2001] introduziram o conceito de proveniência de dados: a capacidade de rastrear a origem e as transformações de um dado dentro de um sistema. Em bancos relacionais, a proveniência pode ser implementada por meio de chaves estrangeiras, registros de auditoria e consultas determinísticas: cada dado pode ser rastreado até sua origem.

3.5. Panorama Tecnológico

Esta subseção apresenta as tecnologias escolhidas para o ThingIndex e como cada uma se relaciona com os fundamentos teóricos discutidos nas subseções anteriores.

Next.js. O frontend utiliza Next.js, framework React mantido pela Vercel. A escolha está alinhada com a necessidade de uma interface que permita ao usuário navegar, consultar e visualizar o conhecimento recuperado. O suporte nativo a Server Components permite renderizar páginas no servidor sem expor chaves de API, e o App Router simplifica a estrutura de rotas e layouts.

Go. A API e os workers de indexação são implementados em Go, linguagem criada na Google em 2009. A escolha foi influenciada pelo pipeline de processamento documental: o parsing e a extração de parágrafos e tabelas (descritos na Seção 6.7) precisam ocorrer de forma concorrente para múltiplos documentos. Go oferece goroutines para concorrência sem a complexidade de threads tradicionais, e a compilação para binário único reduz erros em tempo de execução.

PostgreSQL. O banco de dados principal é o PostgreSQL, sistema relacional maduro. A escolha é uma consequência direta dos fundamentos apresentados na Seção 3.4: o modelo relacional de Codd [Codd 1970], as propriedades ACID e a proveniência de Buneman [Buneman et al. 2001] só são plenamente realizáveis com um banco que ofereça chaves estrangeiras, integridade referencial e índices avançados. O PostgreSQL oferece ainda a extensão pgvector para futuras comparações com abordagens semânticas.

Object Storage. Os arquivos brutos (PDFs) são armazenados em object storage compatível com a API S3. A separação entre armazenamento de objetos e banco relacional é intencional: permite acesso aos arquivos via URLs temporárias (presigned URLs) sem sobrecarregar a API, e mantém o banco com tamanho gerenciável.

Filas Assíncronas (RabbitMQ). O pipeline de indexação é orquestrado por filas de mensagens utilizando RabbitMQ. O padrão outbox é empregado para reduzir o risco de perda de eventos de indexação: quando um documento é persistido no banco, um evento de indexação é registrado na mesma transação. Um dispatcher lê os eventos da tabela de outbox e os publica no RabbitMQ. O RabbitMQ foi escolhido por sua maturidade, suporte a filas de retry e dead-letter queues (DLQ).

Docker Compose. Todo o ambiente de desenvolvimento é reproduzível via Docker Compose, com healthchecks entre serviços, garantindo que o sistema possa ser executado com um único comando independentemente do sistema operacional.

4. Trabalhos Relacionados

Esta seção analisa frameworks e plataformas que atuam em áreas próximas ao ThingIndex. Para cada ferramenta, são descritos sua origem, funcionamento e limitações, destacando as lacunas de pesquisa que o ThingIndex se propõe a preencher.

4.1. Notion

O Notion, criado pela Notion Labs Inc. (fundada por Ivan Zhao e Simon Last em 2013), é um workspace all-in-one que combina documentos, bancos de dados e gerenciamento de tarefas. Sua arquitetura é baseada em blocos aninhados (block-based editor), onde cada elemento (texto, imagem, tabela, banco de dados) é um bloco que pode conter outros blocos. O Notion oferece busca full-text e integração com IA para resumo e geração de texto.

A principal limitação do Notion é a ausência de uma API pública de indexação semântica e a dificuldade de auditar o contexto recuperado. O conhecimento fica preso na plataforma, e não há um pipeline de processamento documental que permita rastrear a origem de cada trecho recuperado.

4.2. LangChain

O LangChain, criado por Harrison Chase em 2022 (LangChain Inc.), é um framework para construção de aplicações baseadas em LLMs. Sua arquitetura é baseada em *chains*: sequências de chamadas a modelos de linguagem, intercaladas com recuperação de informação (*retrievers*), ferramentas externas (*tools*) e agentes autônomos (*agents*). O LangChain se tornou o framework mais popular para prototipação de sistemas RAG, oferecendo integrações prontas com dezenas de provedores de LLM e bancos vetoriais.

A limitação do LangChain está na opacidade do pipeline de recuperação: o desenvolvedor configura um retriever, mas tem pouca visibilidade sobre como o contexto foi segmentado, ordenado e montado. O custo operacional depende de serviços externos (API de embeddings + vector DB + LLM), e a auditabilidade do contexto recuperado é limitada.

4.3. LlamaIndex

O LlamaIndex (originalmente GPT Index), criado por Jerry Liu em 2022, é um framework focado em indexação e recuperação de dados para LLMs. Sua arquitetura suporta múltiplos tipos de índice (lista, vetor, árvore, documento) e permite conectar fontes de dados heterogêneas (PDFs, bancos de dados, APIs). O LlamaIndex se tornou referência para construção de sistemas RAG, oferecendo componentes prontos para ingestão, chunking, embedding e consulta.

A limitação do LlamaIndex está na opacidade do pipeline de recuperação: o desenvolvedor escolhe configurações de chunking e embedding, mas tem baixa visibilidade sobre como o contexto é montado. A dependência de bancos vetoriais torna o custo variável, e a auditabilidade do contexto recuperado é limitada: não há garantia de que cada trecho retornado possa ser rastreado até o documento e página de origem.

4.4. Lacunas de pesquisa

Nenhuma das ferramentas analisadas oferece simultaneamente: (i) pipeline assíncrono de indexação com retry e DLQ; (ii) recuperação de contexto por consultas SQL determinísticas com rastreabilidade total da origem; (iii) desacoplamento de provedores de IA; (iv) custo previsível sem dependência obrigatória de vector DB.

5. Metodologia

Este trabalho caracteriza-se como uma pesquisa aplicada orientada pela Design Science Research (DSR) [Hevner et al. 2004], que estabelece que a construção de um artefato de software e sua avaliação segundo critérios explícitos de utilidade e rigor são inseparáveis da investigação do problema. A pesquisa investiga o problema de fragmentação documental, propõe o artefato ThingIndex como solução, descreve sua arquitetura e define critérios para sua avaliação futura.

Quanto aos procedimentos, o trabalho segue os ciclos da DSR propostos por Hevner et al. [Hevner et al. 2004]: (i) conscientização do problema, (ii) sugestão, (iii) desenvolvimento, (iv) avaliação e (v) conclusão.

A revisão da literatura de Recuperação de Informação (RI) e RAG, apresentada na Seção 3, evidenciou as limitações das abordagens puramente lexicais (matching exato)

e a opacidade dos métodos neurais (embeddings e bancos vetoriais) no que diz respeito à auditabilidade do contexto recuperado. Essa constatação fundamentou a decisão metodológica de adotar a modelagem relacional como baseline primária, priorizando determinismo e rastreabilidade sobre flexibilidade semântica. O desenvolvimento incremental do artefato permitirá avaliar, a cada ciclo, se a abordagem relacional atende aos critérios de precisão e auditabilidade definidos na Proposta.

A classificação como exploratória justifica-se pela investigação de um problema pouco estudado na literatura: o uso de modelagem relacional pura como baseline para inteligência documental em contraste com abordagens vetoriais predominantes. A natureza descritiva se dará na apresentação detalhada da arquitetura, do modelo de dados, dos fluxos de processamento e da estratégia de validação.

As etapas metodológicas do trabalho serão organizadas da seguinte forma:

- levantamento do problema e definição do escopo do sistema;
- revisão bibliográfica dos conceitos que sustentam a proposta;
- levantamento e organização dos requisitos funcionais e não funcionais;
- modelagem da arquitetura e das estruturas de dados;
- implementação incremental do software;
- execução de testes e validação da solução desenvolvida;
- análise dos resultados obtidos.

6. Proposta

Esta seção apresenta o ThingIndex, detalhando sua definição, arquitetura, modelo de dados, fluxo de processamento, tecnologias, protótipo das telas, diagramas, critérios de avaliação e cronograma de desenvolvimento.

6.1. O que é o ThingIndex?

O ThingIndex é uma plataforma open source de inteligência documental baseada em modelagem relacional. Seu objetivo é organizar coleções de documentos PDF, extrair seu conteúdo estrutural (parágrafos, tabelas, metadados) e permitir a recuperação auditável do contexto por meio de consultas SQL determinísticas.

Diferentemente de sistemas RAG convencionais, que dependem de bancos vetoriais e embeddings opacos, o ThingIndex prioriza a rastreabilidade, isto é, cada parágrafo recuperado pode ser rastreado até o documento, página e posição de origem. A arquitetura, detalhada na Seção 6, combina tecnologias para frontend, API, persistência, armazenamento e processamento assíncrono.

A construção de uma plataforma com tais características se justifica pelos fundamentos de inteligência documental e gestão do conhecimento discutidos na Seção 3.1: sem mecanismos de organização e recuperação, o conhecimento explícito acumulado em documentos acadêmicos e organizacionais se perde. O ThingIndex propõe preencher essa lacuna com uma abordagem que privilegia auditabilidade e determinismo sobre flexibilidade semântica, sem excluir a possibilidade de extensões futuras com embeddings ou grafos.

A opção pelo modelo open source não é apenas uma decisão de licenciamento, mas um compromisso com a replicabilidade e a evolução contínua do projeto. Ao tornar

o código fonte acessível, a comunidade acadêmica e técnica poderá examinar, testar e contribuir com extensões, promovendo um ciclo de melhoria que beneficia não apenas o ThingIndex, mas o ecossistema de ferramentas de inteligência documental como um todo. O repositório será mantido público após a conclusão do TCC, permitindo que outros pesquisadores e desenvolvedores realizem forks, submetam contribuições e comparem a abordagem relacional com outras estratégias de recuperação de informação.

6.2. Aplicação dos Fundamentos Teóricos ao ThingIndex

O ThingIndex seguirá a abordagem de pesquisa aplicada descrita por Hevner et al. [Hevner et al. 2004], na qual a construção do artefato de software será inseparável da investigação do problema. O projeto não se limitará a propor uma arquitetura, mas a implementará como monorepo executável, com testes, pipeline de integração e documentação versionada. A organização incremental do código, com autenticação primeiro, depois coleções, depois pipeline assíncrono, refletirá a preocupação com a entrega de valor funcional em cada etapa, coerente com o ciclo de vida iterativo descrito por Pressman e Maxim [Pressman e Maxim 2014].

A arquitetura do ThingIndex adotará separação em camadas com fronteiras explícitas entre interface, aplicação, domínio e infraestrutura. A propriedade mais importante será o desacoplamento: o módulo de autenticação não conhecerá detalhes de storage, o pipeline de indexação não dependerá de um provedor de LLM específico e a recuperação de contexto funcionará independentemente de existir ou não um chat configurado. A aplicação de DDD será pragmática: o domínio conterá contratos e erros de negócio, enquanto adaptadores para serviços externos serão mantidos separadamente. Isso garantirá que a troca de um provedor de armazenamento (de MinIO para GCS, por exemplo) ou de um modelo de linguagem não exija alteração na lógica de coleção ou de recuperação de contexto.

O modelo relacional será a base sobre a qual o ThingIndex construirá sua proposta de auditabilidade. A proveniência descrita por Buneman et al. [Buneman et al. 2001] será materializada literalmente: o contexto de uma coleção poderá ser reconstruído por uma sequência de comandos SQL determinísticos, sem depender de vetores opacos ou de caminhos de inferência não estruturados. A ordem semântica será preservada por índices de ordenação, os vínculos por chaves estrangeiras e o histórico de processamento por tabelas de eventos e tentativas. O sistema optará por consistência transacional (ACID) via PostgreSQL. Essa escolha será deliberadamente conservadora: o sistema aceitará o custo de schema rígido e normalização em troca de previsibilidade operacional, facilidade de auditoria e clareza para fins acadêmicos. Outras abordagens, vetoriais ou de grafo, não serão excluídas, mas relegadas a trilhas comparativas futuras.

A seguir são apresentados os requisitos funcionais e não funcionais do projeto.

6.3. Requisitos Funcionais

- RF01: o sistema deve permitir o cadastro, login, logout e gerenciamento de perfil de usuários;
- RF02: o sistema deve permitir a criação, edição, listagem e exclusão de categorias documentais;
- RF03: o sistema deve permitir a criação, edição, listagem e exclusão de coleções documentais;

- RF04: o sistema deve permitir o envio de documentos PDF com validação de tipo;
- RF05: o sistema deve armazenar documentos em object storage;
- RF06: o sistema deve processar documentos de forma assíncrona, extraindo parágrafos, tabelas, metadados, páginas, chunks e outline;
- RF07: o sistema deve permitir enriquecimento opcional por LLM durante a indexação;
- RF08: o sistema deve possibilitar a recuperação de contexto completa de uma coleção via API REST;
- RF09: o sistema deve oferecer chat grounded por coleção com transmissão via SSE e citação de fontes;
- RF10: o sistema deve permitir configuração de provedores externos (LLM e storage) com criptografia de credenciais;
- RF11: o sistema deve fornecer painel de monitoramento com telemetria por documento e agregada por usuário;
- RF12: o sistema deve expor benchmark público de indexação com corpus controlado;
- RF13: o sistema deve registrar trilha de auditoria para operações em documentos e provedores;
- RF14: o sistema deve permitir retry manual de documentos falhos e detecção de documentos órfãos;
- RF15: o sistema deve suportar internacionalização (PT-BR e EN-US) nas respostas da API.

6.4. Requisitos Não Funcionais

- RNF01: o sistema deve apresentar interface responsiva e navegação clara;
- RNF02: o sistema deve garantir integridade referencial via chaves estrangeiras e consistência transacional (ACID) no PostgreSQL;
- RNF03: o sistema deve suportar documentos e coleções com centenas de documentos sem degradação significativa;
- RNF04: o sistema deve permitir manutenção e evolução incremental, com separação de camadas e fronteiras de domínio explícitas;
- RNF05: o sistema deve adotar mecanismos de segurança: hash para senhas, tokens para autenticação, criptografia para credenciais, rate limiting em rotas sensíveis e sanitização de entradas;
- RNF06: o sistema deve ser reproduzível via Docker Compose, com ambiente de desenvolvimento configurável em máquinas Linux, macOS e Windows (WSL);
- RNF07: o sistema deve possuir cobertura de testes automatizados em nível unitário, integração e end-to-end;
- RNF08: o sistema deve gerar documentação da API automaticamente via Swagger/OpenAPI a partir de anotações no código.

6.5. Visão Geral da Arquitetura

O sistema será organizado em quatro camadas principais: interface de usuário, API, persistência e infraestrutura auxiliar (object storage, filas), cujas tecnologias são detalhadas na Seção 3.5. A Figura 3 ilustrará essa composição. Richards [Richards 2015] classifica essa organização como uma variação do padrão *Layered Architecture*, no qual cada camada possui responsabilidade bem definida e a dependência flui em uma única direção.

A API será a peça central. Ela exporá rotas HTTP para autenticação, gerenciamento de coleções, upload de documentos, configuração de provedores, recuperação de contexto e chat. As rotas serão organizadas em *handlers* finos, que apenas validarão entrada, autenticarão e serializarão respostas. A lógica de negócio residirá em *use cases*, que orquestrarão chamadas a *repositories* e serviços auxiliares.

A Figura 1 apresenta a visão geral da arquitetura do ThingIndex, consolidando a integração entre frontend, API, banco de dados, filas e provedores de IA.

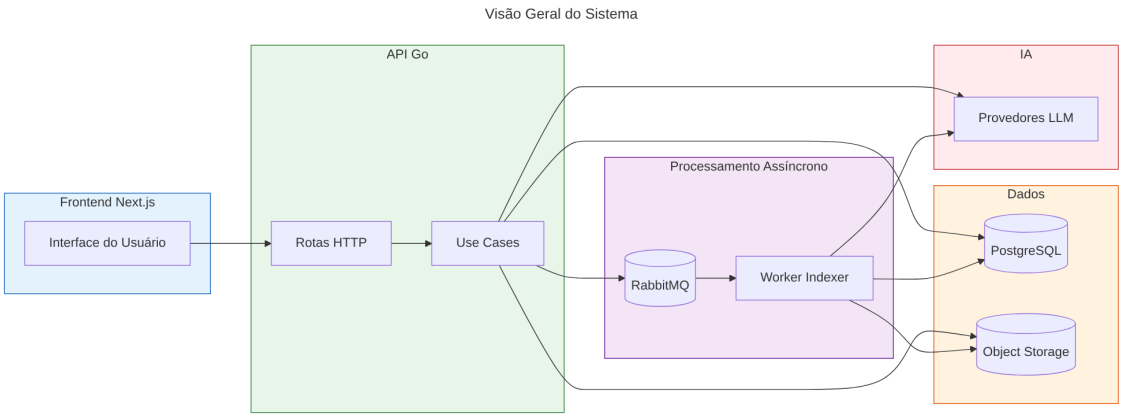


Figura 1. Visão geral da arquitetura do sistema.

A Figura 2 apresenta o diagrama de casos de uso do ThingIndex, com os atores e as funcionalidades disponíveis no sistema.

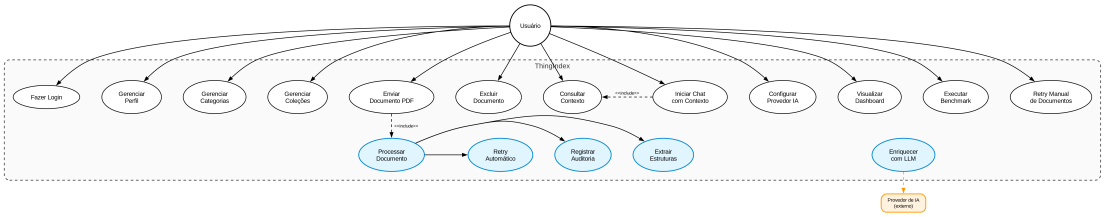


Figura 2. Diagrama de casos de uso do ThingIndex.

O pipeline assíncrono será implementado por meio do padrão *outbox*: quando um documento for enviado, um evento será gravado na mesma transação do banco que persistir o documento. Um processo *dispatcher* realizará polling na tabela de outbox e publicará os eventos no RabbitMQ. Essa estratégia reduz o risco de perda de mensagens: se a API cair antes de publicar no broker, o evento ainda estará no banco e será reenviado na próxima rodada do dispatcher. Richardson [Richardson 2018] descreve esse padrão como a solução mais robusta para o problema de dualidade de escrita (banco + broker) em arquiteturas baseadas em microsserviços.

O worker *indexer* consumirá a fila, processará o documento e persistirá as estruturas extraídas. Se o processamento falhar, o evento será reenviado para uma fila de *retry* com atraso configurável. Após esgotar as tentativas, ele será movido para uma *dead-letter queue* (DLQ), onde poderá ser inspecionado manualmente. Hohpe e Woolf [Hohpe e Woolf 2003] classificam esses mecanismos como *Dead Letter Channel* e

Resequencer, padrões consolidados de integração empresarial que permitem entrega confiável mesmo em cenários de falha parcial.

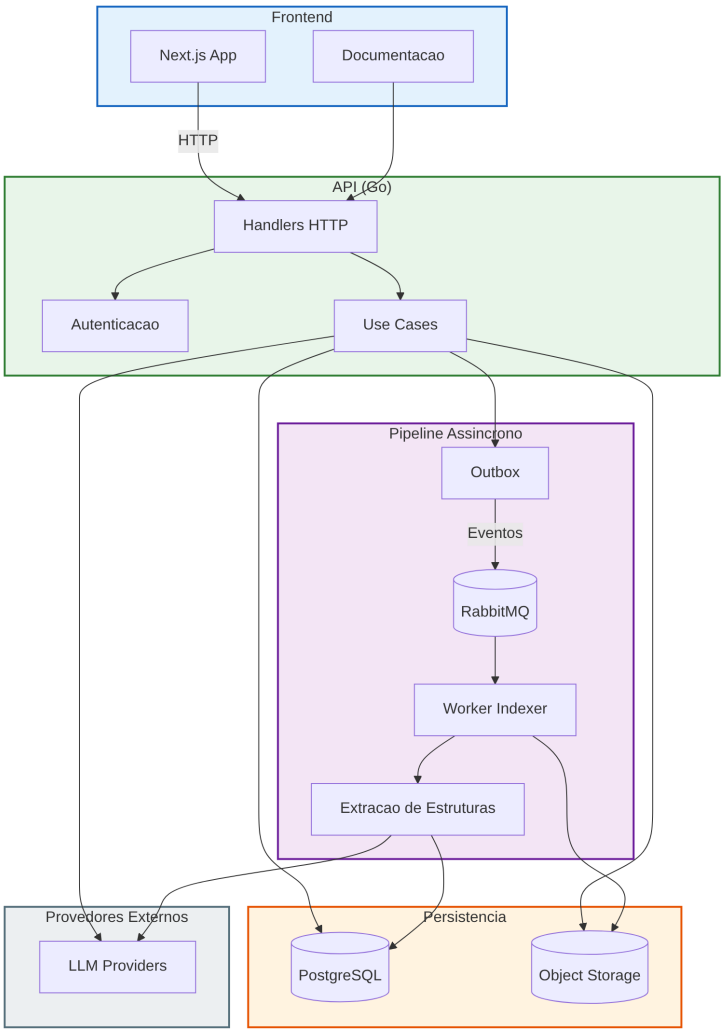


Figura 3. Arquitetura runtime do ThingIndex, com separação entre camada web, API, persistência e processamento assíncrono.

6.6. Modelagem de Dados

O modelo de dados será projetado para que o contexto de uma coleção seja reconstruído por consultas SQL determinísticas. O esquema completo contempla 20 tabelas organizadas em três grupos lógicos: núcleo, estruturas extraídas e infraestrutura, conforme descrito a seguir e disponível na íntegra no Apêndice A. A Figura 4 apresenta o diagrama entidade-relacionamento do grupo principal (núcleo), com as entidades fundamentais do sistema.

A Figura 4 apresenta o diagrama entidade-relacionamento do núcleo do sistema, composto pelas entidades fundamentais: usuários, categorias, coleções e documentos. Esse núcleo estabelece as relações básicas sobre as quais todo o sistema se apoia. Um `user` pode possuir múltiplas `categories` e `collections`, e cada `collection` agrupa diversos `documents`. A `collection` pode opcionalmente pertencer a uma

`category`, permitindo organização hierárquica. Cada `document` armazena metadados como caminho, tipo MIME, tamanho e status de processamento.



Figura 4. Diagrama entidade-relacionamento do núcleo: usuários, categorias, coleções e documentos.

A partir do documento, as estruturas extraídas são fragmentadas em seis tipos de tabelas, cada uma responsável por um aspecto específico do conteúdo:

- páginas: conteúdo textual de cada página, com resumo e estimativa de tokens;
- parágrafos: blocos textuais com posição sequencial e número da página;
- tabelas: conteúdo tabular com cabeçalhos e linhas;
- metadados: pares chave-valor associados ao documento;
- unidades de recuperação: trechos com tipo, título e vínculo aos parágrafos de origem;
- hierarquia de seções: estrutura de tópicos com níveis e referência ao pai.

Além das entidades de domínio, o esquema inclui tabelas de infraestrutura: eventos de outbox para o padrão de mensageria, tentativas de processamento para rastreabilidade de falhas, eventos de auditoria para trilha de operações, configurações de provedores

com credenciais criptografadas, conversas e mensagens de chat, além de dados de precificação para rastreio de custos.

A fragmentação intencional do conteúdo em tabelas separadas evita colunas polimórficas ou JSONB monolítico, garantindo que cada tipo de dado tenha seu próprio esquema bem definido. Índices compostos e chaves estrangeiras preservam a ordenação original e a integridade referencial entre as tabelas.

A Figura 5 complementa essa visão com o diagrama de classes, que organiza as 14 entidades nos três grupos lógicos. O diagrama de classes permite visualizar não apenas as tabelas, mas também os métodos associados a cada entidade e os relacionamentos com suas cardinalidades, oferecendo uma perspectiva orientada a objetos do modelo de dados.

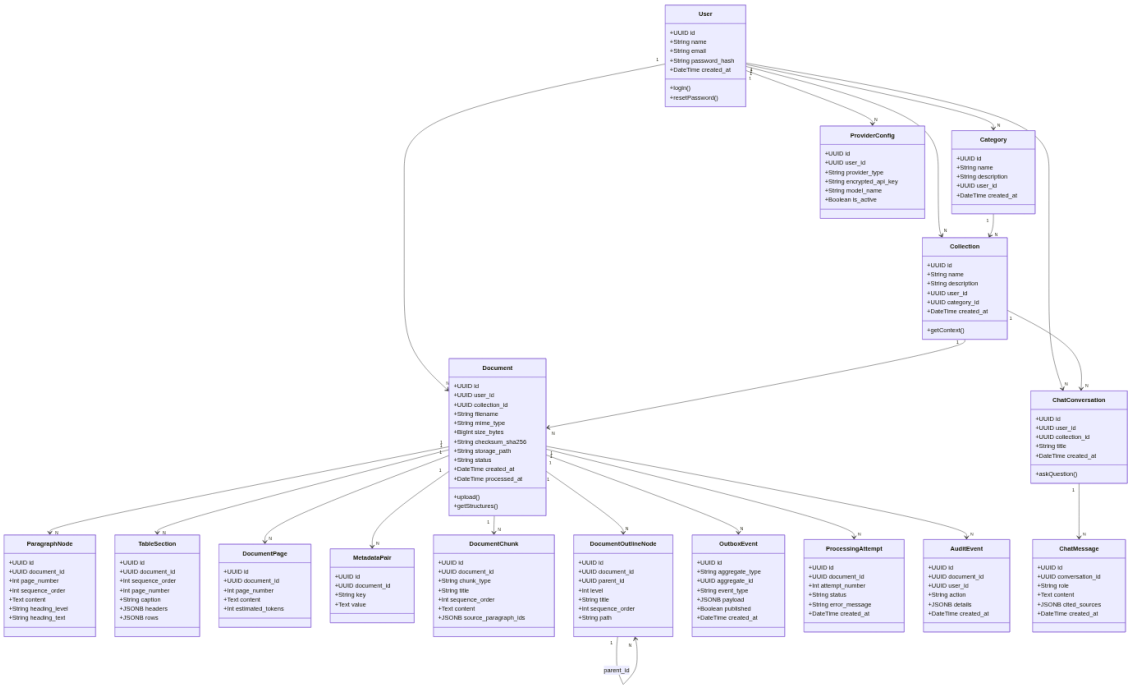


Figura 5. Diagrama de classes do ThingIndex, com 14 classes organizadas em núcleo (azul), estruturas (verde) e infraestrutura (roxo).

6.7. Fluxo de Processamento

O fluxo de processamento de um documento no ThingIndex pode ser dividido em três etapas principais, cada uma com responsabilidades bem definidas dentro da arquitetura do sistema.

A primeira etapa compreende o upload e a validação do documento. Quando o usuário envia um arquivo PDF, a API verifica o tipo MIME e a existência da coleção de destino. O arquivo é armazenado em object storage e registrado no banco de dados com status pendente. Nesse momento, um evento de indexação é inserido em uma tabela de outbox na mesma transação, garantindo que nenhum documento fique sem processamento mesmo em caso de falha da API.

A segunda etapa é a indexação assíncrona, executada por um worker dedicado. O dispatcher lê os eventos da tabela de outbox e os publica no RabbitMQ. O worker

indexer consome a mensagem e realiza o parsing do PDF, extraindo parágrafos, tabelas e metadados. Opcionalmente, um LLM pode enriquecer o conteúdo extraído. Todas as estruturas são persistidas no banco relacional, e a telemetria do processamento é registrada para auditoria. Em caso de falha, o evento é reenviado para uma fila de retry e, após esgotar as tentativas, movido para uma dead-letter queue para inspeção manual.

A terceira etapa é a recuperação de contexto. Quando uma consulta é realizada, o sistema executa joins SQL sobre as tabelas tipadas, monta um JSON hierárquico com todo o contexto da coleção e disponibiliza o resultado para o chat com citação de fontes.

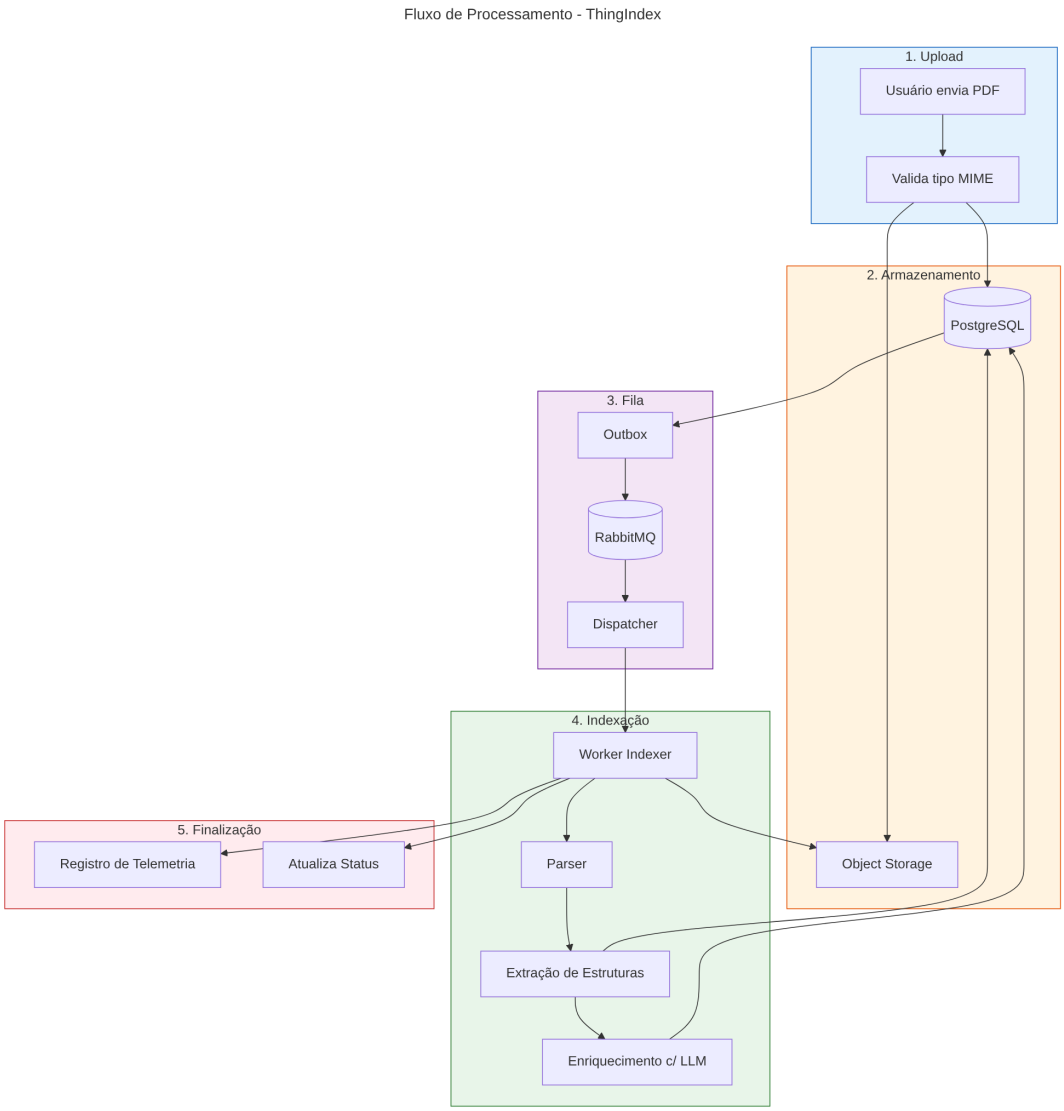


Figura 6. Fluxo completo: upload, enfileiramento, indexação e recuperação de contexto.

6.7.1. Upload e Indexação

O usuário autenticado envia um arquivo PDF. A API valida o tipo MIME e a existência da coleção de destino. O arquivo é enviado ao object storage e registrado no PostgreSQL

com status pendente. Um evento de indexação é inserido na tabela de outbox na mesma transação.

O dispatcher publica o evento no RabbitMQ. O worker indexer consome a mensagem e executa: leitura do arquivo, parsing, extração de parágrafos e tabelas, enriquecimento opcional por LLM, persistência das estruturas e registro de telemetria. Em caso de falha, o evento segue para retry ou dead-letter queue. A Figura 6 ilustra esse fluxo completo.

6.7.2. Interação entre Componentes

Para complementar a visão do fluxo de processamento, o diagrama de sequência apresentado a seguir mostra a ordem temporal das chamadas entre os componentes do sistema. Nele é possível visualizar como o frontend se comunica com a API, como a API interage com o banco de dados e o storage, como o evento de indexação é propagado para o RabbitMQ e consumido pelo worker, e finalmente como o contexto é recuperado e exibido ao usuário.

A Figura 7 detalha essa comunicação durante os três fluxos principais, abrangendo desde o upload do documento até a resposta do chat com citação de fontes.

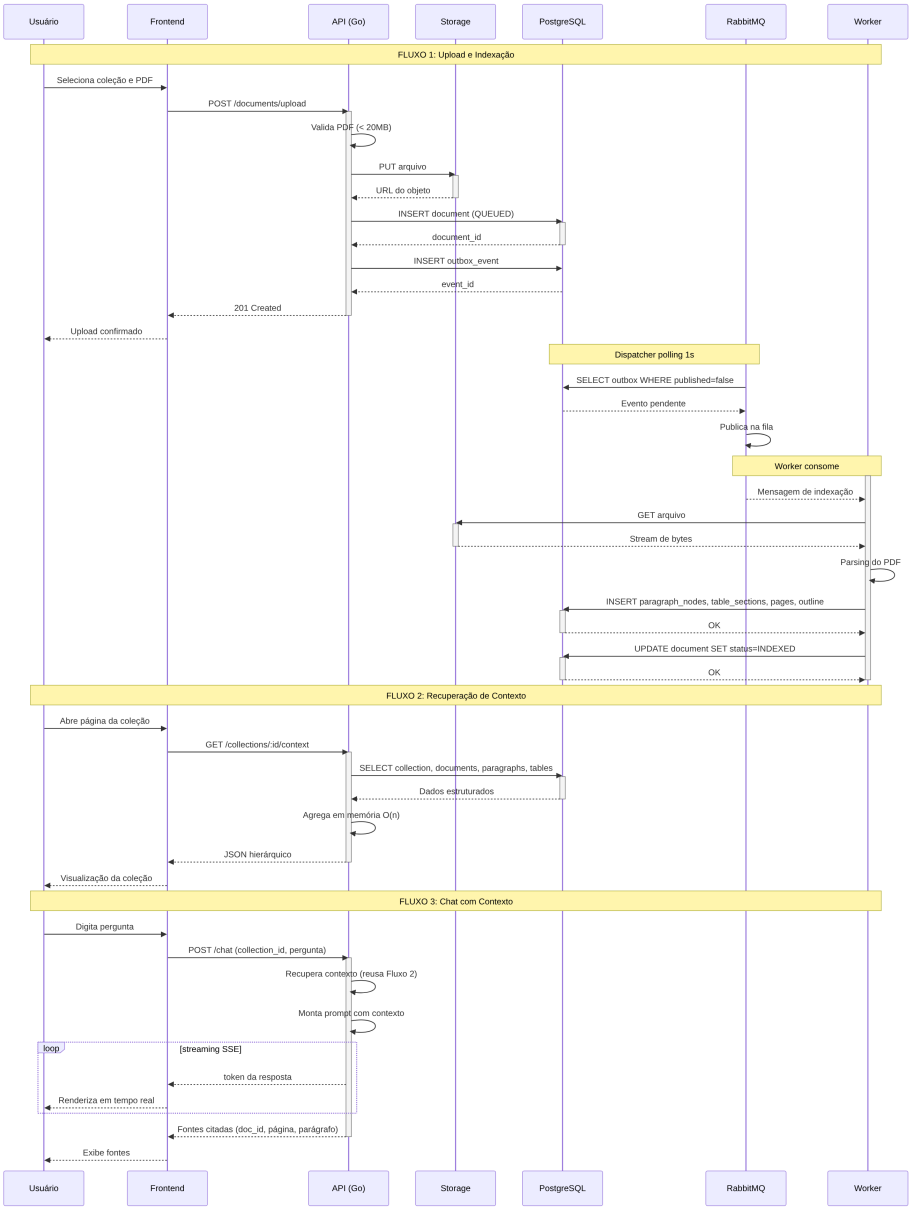


Figura 7. Diagrama de sequência do ThingIndex, ilustrando a comunicação entre frontend, API, storage, banco, fila e worker.

6.8. Prototipação de Telas

Esta subseção apresenta os protótipos de tela do ThingIndex, ilustrando a interface proposta para o sistema.

6.8.1. Landing Page

A landing page apresenta a proposta de valor do ThingIndex. A Figura 8 ilustra essa tela.



Figura 8. Página inicial do ThingIndex.

6.8.2. Dashboard

O dashboard operacional oferece gerenciamento de categorias e coleções, listagem de documentos com preview lateral e telemetria, upload por arrastar-e-soltar, painel de monitoramento agregado e tela de chat com histórico de conversas e fontes consultadas. A Figura 9 ilustra essa tela.

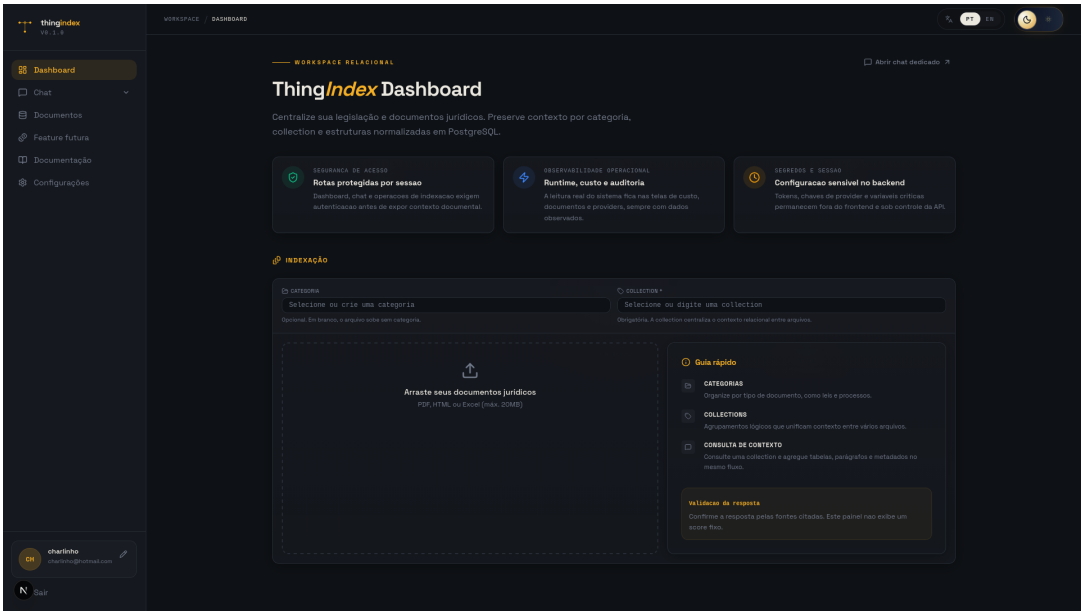


Figura 9. Dashboard operacional.

6.8.3. Documentos

A tela de documentos permite visualizar e gerenciar os arquivos enviados, com informações de status de processamento e acesso ao conteúdo extraído. A Figura 10 ilustra essa tela.

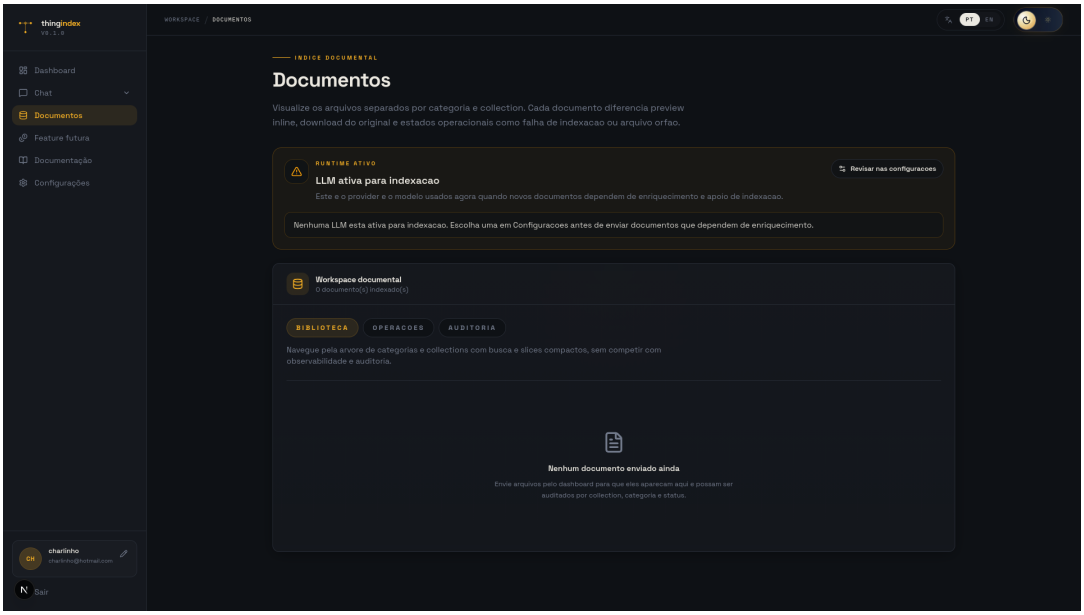


Figura 10. Tela de gerenciamento de documentos.

6.8.4. Configurações

A tela de configurações permite ao usuário gerenciar provedores de IA e storage, com criptografia de credenciais no backend. O usuário poderá definir provedores distintos para indexação e chat. A Figura 11 ilustra essa tela.

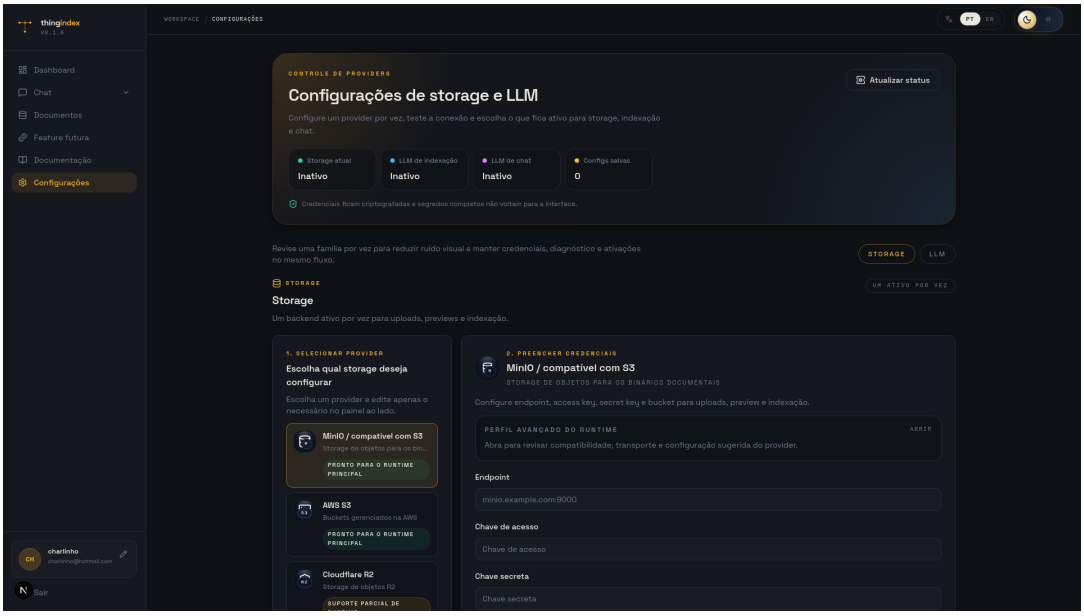


Figura 11. Tela de configurações de provedores.

6.9. Avaliação da Qualidade

Para validar se o ThingIndex atende aos objetivos propostos, serão aplicados critérios de avaliação divididos em três dimensões: precisão na recuperação, auditabilidade e custo operacional apresentadas a seguir.

6.9.1. Precisão na Recuperação de Contexto

A precisão será medida por meio de testes funcionais com corpus controlado. Um conjunto de documentos PDF com conteúdo conhecido será inserido no sistema, e consultas de contexto serão realizadas sobre coleções específicas. A métrica principal será a taxa de acerto: para cada consulta, verificar se todos os documentos relevantes da coleção foram retornados e se a ordem dos parágrafos e tabelas foi preservada.

O experimento consistirá em:

1. Inserir N documentos PDF com estrutura conhecida (parágrafos numerados, tabelas identificadas);
2. Executar consulta de contexto para cada coleção;
3. Comparar o JSON retornado com o conteúdo original, verificando:
   - presença de todos os documentos da coleção;
   - ordem correta dos parágrafos;
   - integridade dos metadados extraídos;
   - correspondência entre fontes citadas e parágrafos de origem.
4. Calcular a proporção de acertos sobre o total de elementos esperados.

6.9.2. Auditabilidade

A auditabilidade será verificada rastreando a origem de cada parágrafo recuperado. Para cada elemento do contexto retornado, o sistema deve ser capaz de informar: documento

de origem, número da página, posição sequencial e tipo de estrutura (parágrafo, tabela, metadado). O teste consiste em percorrer o JSON de resposta e confirmar que todos os campos de proveniência estão preenchidos e apontam para registros existentes no banco.

6.9.3. Custo Operacional

O custo será medido em duas categorias: (i) custo de armazenamento por documento no object storage e no PostgreSQL; (ii) custo de processamento por documento (tempo de CPU, memória e chamadas a LLM, quando aplicável). O objetivo é demonstrar que a baseline relacional tem custo previsível e independente de serviços externos de vector DB.

6.10. Cronograma

O desenvolvimento do ThingIndex será realizado entre junho e dezembro de 2026, conforme ilustrado na Figura 12.

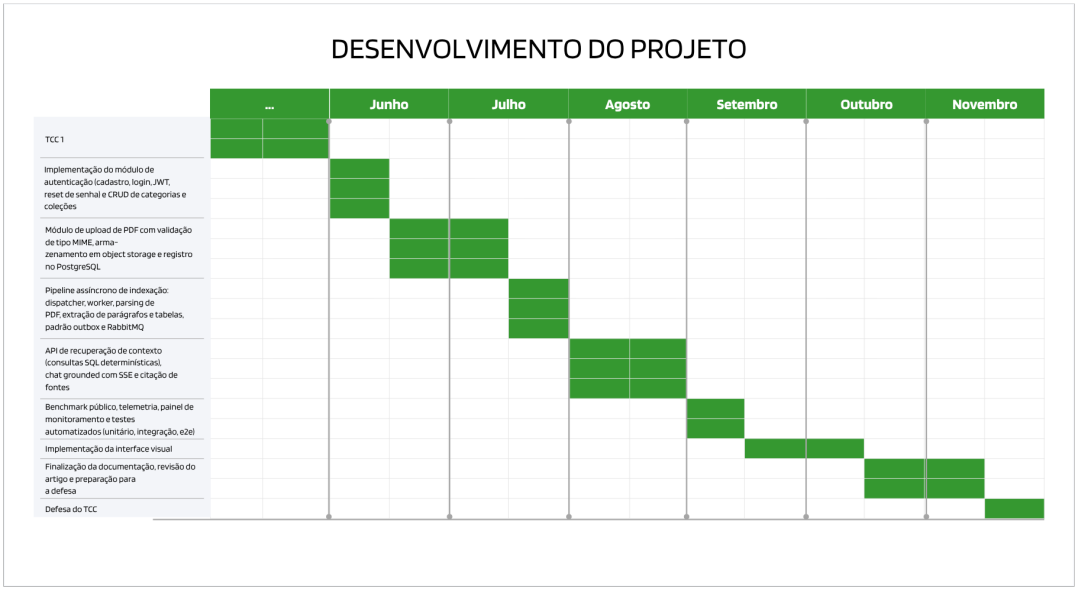


Figura 12. Cronograma de desenvolvimento do ThingIndex.

7. Considerações Finais

Este artigo apresentou a proposta do ThingIndex, uma plataforma de inteligência documental baseada em modelagem relacional, concebida para organizar, processar, indexar e recuperar contexto documental com auditabilidade e potencial de evolução. A partir da fundamentação teórica, da definição metodológica, do levantamento de requisitos e da modelagem arquitetural realizados neste trabalho, foi possível consolidar as bases conceituais e estruturais necessárias para o desenvolvimento da plataforma na etapa seguinte da pesquisa.

Como contribuição do TCC I, destaca-se a formulação de uma hipótese de investigação clara: a de que uma baseline relacional pode atender a cenários de inteligência

documental frequentemente associados a soluções baseadas em vetores ou grafos. Essa hipótese não rejeita a utilidade de técnicas de IA, mas propõe que uma base determinística, testável e de custo previsível seja considerada como alternativa ou complemento às abordagens mais usuais.

A validação dessa hipótese, bem como a implementação efetiva da arquitetura proposta, será efetuada no TCC II, quando serão desenvolvidos o protótipo, os testes de desempenho e a avaliação prática da plataforma frente a cenários documentais estruturados. Espera-se, com isso, que o ThingIndex venha a representar uma contribuição técnica e acadêmica relevante, ainda que sua validação completa dependa das etapas futuras de desenvolvimento e experimentação.

Declaração do uso de ferramentas de IA Generativa no processo de escrita

No desenvolvimento deste manuscrito, os autores utilizaram o serviço ChatGPT, baseado no modelo de linguagem GPT-4, especificamente para fins de revisão textual. Após aplicação desta ferramenta, os autores revisaram e refinaram meticulosamente o conteúdo conforme necessário. É imperativo destacar que os autores assumem total responsabilidade pelo conteudo final da publicação.

Referências

Buneman, P., Khanna, S., e Tan, W.-C. (2001). Why and where: A characterization of data provenance. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.

Date, C. J. (2004). *An Introduction to Database Systems*. Addison-Wesley, Boston, 8 edition.

Davenport, T. H. e Prusak, L. (1998). *Working Knowledge: How Organizations Manage What They Know*. Harvard Business School Press, Boston.

Devlin, J., Chang, M.-W., Lee, K., e Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics.

Elmasri, R. e Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson, Boston, 7 edition.

Hevner, A. R., March, S. T., Park, J., e Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–105.

Hohpe, G. e Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, Boston.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Dai, W., Madotto, A., e Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., e Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

Manning, C. D., Raghavan, P., e Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Mikolov, T., Chen, K., Corrado, G., e Dean, J. (2013). Efficient estimation of word representations in vector space. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Nonaka, I. e Takeuchi, H. (1995). *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, New York.

Pressman, R. S. e Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, 8 edition.

Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media, Sebastopol.

Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications, Shelter Island.

Robertson, S. e Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Salton, G. (1989). *Automatic Text Processing: The Transformation, Analysis, and Retrieval of Information by Computer*. Addison-Wesley, Reading.

Schymik, G., Corral, K., Schuff, D., e St Louis, R. (2015). The benefits and costs of using metadata to improve enterprise document search. *Decision Sciences*, 46(6):1049–1075.

Apêndices e Anexos

Apêndice A: Esquema Completo do Banco de Dados

O esquema relacional completo do ThingIndex, contendo todas as 20 tabelas, tipos de dados, chaves primárias, chaves estrangeiras e índices, está disponível em formato DBML no repositório do projeto. Os arquivos podem ser importados no `dbdiagram.io` para visualização interativa.